

Argus: Sistema de Monitoreo del Conductor Basado en Visión Artificial para la Prevención de Accidentes por Fatiga en el Autotransporte de Carga

Erick Alejandro Carrillo López, Paulino Jacob Suárez Ortiz, Rafael Agustín Pulido Tobías, Mario Antonio Ruz Canul

CENTRO UNIVERSITARIO DE CIENCIAS

EXACTAS E INGENIERÍAS, (CUCEI, UDG)

erick.carrillo4982@alumnos.udg.mx

Paulino.suarez8804@alumnos.udg.mx

rafael.pulido4119@alumnos.udg.mx

[correo del asesor – completar]

FIRMA DE VISTO BUENO DEL ASESOR

Resumen— El autotransporte de carga mueve el 57% de las mercancías de México, pero la fatiga y somnolencia causan entre el 24% y 30% de los accidentes viales, agravado por el incumplimiento generalizado de la NOM-087. Argus es un sistema de monitoreo del conductor (DMS) que aumenta, sin reemplazar, al operador: una capa preventiva que detecta somnolencia en tiempo real mediante visión artificial y actúa antes de un microsueño. El sistema combina un módulo de inferencia en el borde (Raspberry Pi 5) con un microcontrolador (ESP32) responsable de alertas, frenado preventivo y comunicación, y un buffer local que garantiza continuidad sin cobertura celular. El modelo de IA es un clasificador binario (Not Drowsy / Drowsy) que fusiona, por cada instante, el embedding de una red convolucional (CNN) congelada con un subconjunto de características geométricas faciales, y clasifica esa secuencia con una red LSTM sobre una ventana temporal. Alcanzó 84.24% de exactitud y 0.8375 de F1 macro en sujetos de prueba nunca vistos en entrenamiento, casi el doble del F1 de una CNN de un solo frame (0.5273) sobre los mismos sujetos.

Palabras clave – monitoreo del conductor, somnolencia, visión artificial, CNN, LSTM, MediaPipe, sistemas embebidos, sistemas distribuidos.

I. INTRODUCCIÓN

México depende del autotransporte federal para mover más de la mitad de su carga, pero paga un costo altísimo en vidas: cerca de 17,000 muertes al año por accidentes de transporte terrestre, de los cuales la fatiga del conductor explica hasta un 30%. El ciclo es conocido: jornadas «justo a tiempo» que exceden la NOM-087, bitácoras falsificadas, y un 81.3% de los operadores recurriendo a sustancias para mantenerse despiertos. La automatización total (Nivel 5) — el camino que ya toman flotas comerciales en EUA — no es viable en México a corto plazo: un camión detenido por su protocolo de «minimal risk condition» ante una anomalía es un blanco fácil para el robo de carga, la red carretera carece de la señalización y cobertura 5G que la conducción autónoma exige, y la brecha de costo (450,000 USD vs. 180,000 USD) es prohibitiva para el «hombre-camión» que domina el mercado. El sistema toma su nombre de Argos Panoptes, el gigante centinela de la mitología griega que nunca dormía por completo: vigilaba con unos ojos mientras otros descansaban. Argus busca esa misma

vigilancia constante para el conductor — no reemplazarlo, sino ser los ojos que no se cierran cuando los suyos lo hacen. El sistema propone «Aumento Humano»: una barrera tecnológica de bajo costo que vigila el estado fisiológico del conductor y solo interviene — alertando o frenando preventivamente — cuando el humano ya no puede reaccionar a tiempo.

II. TRABAJOS RELACIONADOS

A. Datasets y estudios sobre microsueños

RLDD — dataset público de 60 sujetos (~30 h de video) etiquetado en tres niveles de vigilancia, con un modelo temporal (LSTM jerárquico) sobre features de parpadeo [1]. Valida el enfoque temporal usado en este proyecto y es candidato para ampliar el entrenamiento.

- UL-DD — dataset multimodal de 19 sujetos con video, señales biométricas y sensor de presión en el volante [2] — el mismo tipo de sensor que Argus contempla. Referencia natural para un futuro módulo de fusión biométrica.

- Un estudio en simulador con 144 sujetos muestra que rasgos de personalidad e IQ influyen en cuándo aparece el microsueño de cada persona [3], evidencia de que un umbral fijo único (tipo PERCLOS) no generaliza bien entre individuos — lo que refuerza preferir un modelo estadísticamente validado por característica.

B. Proyectos con el mismo problema, enfoque distinto

Un trabajo de grado que detecta somnolencia diurna y nocturna con una CNN MobileNetV3 sobre 12 datasets públicos reporta 97% de precisión detectando ojos cerrados [4]. Es un clasificador de visión acotado, sin backend, comunicación entre dispositivos ni sistema de alertas/actuación. Argus se diferencia en dos ejes: clasifica secuencias (embedding de CNN fusionado con features geométricas, LSTM sobre ventanas), no frames sueltos, y extiende el problema hacia un sistema distribuido completo (borde + alerta + frenado + nube).

- Soluciones comerciales de video-telemática (Geotab [5], Samsara [6]) ya despliegan monitoreo del conductor a nivel flota; Argus se diferencia por ser edge-first, pensado para conectividad intermitente y el riesgo de robo de carga en México.

- La automatización comercial de Nivel 4 en EUA (Aurora [7], Kodiak [8]) sirve de contraste directo con la inviabilidad de la automatización total en el contexto mexicano descrita en la Sección I.

III. DESCRIPCIÓN DEL DESARROLLO DEL PROYECTO MODULAR

A. Metodología de trabajo

El equipo, integrado por tres personas, trabajó bajo Scrum: un backlog priorizado de requerimientos llevaba el registro de las piezas pendientes, en progreso y terminadas, y cada semana se realizó una reunión de avance para revisar lo completado y asignar las siguientes tareas.

B. Requerimientos principales y backlog

La Tabla I resume el backlog priorizado de requerimientos del MVP de hardware/firmware y su estado de avance a la fecha de este documento.

TABLA I
ESTADO DEL BACKLOG DEL MVP

#	Requerimiento	Estado
1	Detección facial y ocular (visión artificial)	Completado — corriendo en vivo en cv-argus
2	Alertas sonoras en cabina ante microsueño	Completado
3	Frenado autónomo preventivo (CAN/AEB)	Completado — señal simulada, sin actuador físico conectado
4	Transmisión de alertas/ubicación por red celular	Completado — envío de alerta real, sin geolocalización en vivo todavía
5	Memoria local (buffer), reenvío automático	Completado
6	Botón de pánico	Pendiente — depende del firmware ESP32

La Tabla II resume además el backlog de modelado, diseño y desarrollo de software, complementario al del MVP de hardware.

TABLA II
BACKLOG DE MODELADO, DISEÑO Y DESARROLLO DE SOFTWARE

#	Tarea	Estado
1	Diagrama de arquitectura del sistema (borde + nube)	Completado
2	Diagrama del pipeline del modelo (MediaPipe + CNN + LSTM)	Completado
3	Diagrama ER de la base de datos	Completado
4	Creación del dataset (landmarks/features geométricas)	Completado
5	Entrenamiento y comparación de modelos	Completado

	(RandomForest, red densa, CNN, CNN+LSTM)	
6	Selección del umbral de decisión del modelo final	Completado
7	Integración del modelo final en el módulo de borde (cv-argus)	Completado
8	Backend (FastAPI + MongoDB): usuarios, camiones, conductores, alertas, rutas	Completado
9	Frontend (React): login y pantalla de operación en vivo	Completado — otras pantallas con datos de prueba
1	Pruebas de integración end-to-end (Playwright)	Completado
0		

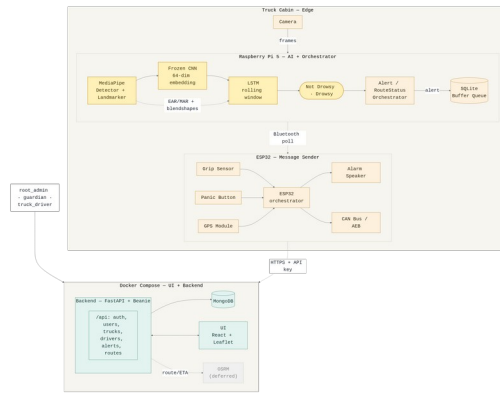


Fig. 1 — Arquitectura del sistema Argus: borde (Raspberry Pi 5 + ESP32) y nube (FastAPI + MongoDB + React).

C. Tecnologías utilizadas

Modelo/IA: Python, MediaPipe (Face Detector para el recorte, FaceLandmarker para las features geométricas), TensorFlow/Keras (CNN + GeometricRatioFeatureLayer propia + LSTM sobre el embedding fusionado), scikit-learn (RandomForest como baseline).

- Borde (Raspberry Pi 5): contenedor Docker, picamera2 para la cámara CSI, descarga de los modelos entrenados desde almacenamiento en la nube al construir la imagen.

- Microcontrolador (ESP32): C (Arduino/ESP-IDF), con parseo GPS/NMEA por UART.

- Persistencia local: SQLite en modo WAL, para lectura/escritura concurrente entre el proceso que encola alertas y el que las despacha.

- Backend/nube: FastAPI con MongoDB (vía Beanie); OSRM para cálculo de rutas queda como mejora futura.

- Frontend: React con react-leaflet, mostrando geolocalización y estado de flota/alertas.

D. Arquitectura y diseño del sistema

La Fig. 1 muestra la arquitectura completa del sistema: en el borde, la Raspberry Pi 5 ejecuta MediaPipe y el modelo fusionado CNN+LSTM y decide el estado del conductor, comunicándolo por Bluetooth al ESP32, que actúa (alarma, freno preventivo, botón de pánico) y habla por HTTP con el backend; en la nube, FastAPI + MongoDB sirven a un frontend React con mapa en tiempo real.

E. Pruebas realizadas

Se comparó un conjunto de arquitecturas sobre el mismo problema binario (Not Drowsy / Drowsy), resumido en la Tabla III: RandomForest y una red densa sobre features de un solo frame quedaron topadas en 33–41% de accuracy (correlación de Spearman máxima $|r|=0.26$ entre cualquier feature de un solo frame y el nivel de somnolencia); una CNN sobre el recorte facial completo subió a 59.64% / 0.5273 F1 macro; y el modelo final — el embedding congelado de esa CNN fusionado con features geométricas y alimentado a una LSTM sobre una ventana de hasta 100 frames — alcanzó 84.24% accuracy / 0.8375 F1 macro. División train/val/test agrupada por sujeto (StratifiedGroupKFold) en todos los casos.

Modelo	Acc.	F1	Rec. D.
RandomForest (7 feat.)	32.6%	—	0.13
Red densa (58 feat.)	38.6–40.8%	—	—
CNN 1 frame	59.64%	0.5273	—
CNN+LSTM (cero)	62.29%	0.6078	0.47
CNN+LSTM congelado*	84.24%	0.8375	0.73
Ensemble	82.99%	0.8255	0.73

F. Proceso de implementación

Pipeline de notebooks encadenados por etapa (Colab + Drive): extracción de recortes faciales (Face Detector) → extracción de features geométricas sobre esos recortes (FaceLandmarker) → entrenamiento de la CNN → congelamiento de su embedding y fusión con las features geométricas por instante

→ entrenamiento de la LSTM sobre esa secuencia fusionada → exportación del modelo final y carga en cv-argus para inferencia en vivo en el Pi, con una ventana deslizante en memoria (vectores ya fusionados, no imágenes) que se actualiza un frame a la vez.

Módulo I Gestión de la tecnología de información

1.1 Lenguajes: Python en todo el pipeline de IA (ecosistema maduro de MediaPipe/TensorFlow/OpenCV). C en el firmware ESP32 (estándar para ese microcontrolador; suficiente para GPS por UART y control de GPIO).

- 1.2 Bases de datos / estructuras de datos: SQLite como cola de buffer local, sin servidor, modo WAL para concurrencia productor/consumidor; una ventana circular numpy de forma (100, 74) como buffer del modelo desplegado (64 del embedding CNN + 10 geométricas por instante, ~30 KB, sin reprocesar imágenes); MongoDB como base de datos del backend, esquema flexible por documento para users/trucks/drivers/alerts/routes.

- 1.3 Metodología: Scrum (ver Sección III-A).

- 1.4 Ingeniería de software: separación de responsabilidades explícita en cv-argus (model/, pipeline/, orchestrator/, buffer/, sender/, alerts/) con dependencias en una sola dirección. Docker como mecanismo de reproducibilidad.

- 1.5 Modelado del sistema: la Fig. 1 traza una frontera explícita entre lo que decide (Raspberry Pi, IA) y lo que actúa (ESP32, hardware de seguridad), y entre el borde (tiempo real, crítico) y la nube (gestión, no crítica en tiempo real).

Módulo II Sistemas robustos, paralelos y distribuidos

El mérito distribuido del proyecto no se apoya únicamente en el tramo hacia la nube — un backend centralizado consumido por clientes vía HTTP no constituye, por sí solo, un sistema distribuido — sino en elementos concretos de cómputo y comunicación entre dispositivos del propio equipo.

- Procesamiento distribuido: la inferencia del modelo CNN+LSTM sobre video ocurre en el borde, dentro de cada camión, en vez de enviar video crudo a un servidor central. La

carga de cómputo está descentralizada por diseño, no solo por la escala de la flota.

- Comunicación directa entre dos dispositivos: la Raspberry Pi 5 (decide) y el ESP32 (actúa) se comunican por Bluetooth, con el ESP32 haciendo pull periódico sobre la cola SQLite de la Pi — un protocolo propio implementado en ambos extremos, no el consumo de un servicio cliente/servidor ya construido por terceros.

- Tolerancia a fallos con buffer local: la cola SQLite persiste las alertas cuando no hay conectividad y las reenvía automáticamente al recuperar la señal.

- Concurrencia real: el proceso que encola y el que despacha alertas operan de forma concurrente sobre el mismo archivo SQLite en modo WAL.

Módulo III Justificación de Cómputo Flexible (softcomputing)

Ramas de IA cubiertas: redes neuronales (CNN + LSTM), aprendizaje automático (RandomForest como baseline) y visión artificial (MediaPipe Face Detector + FaceLandmarker).

- Modelo matemático: la capa GeometricRatioFeatureLayer calcula, por frame, EAR (Eye Aspect Ratio) y MAR (Mouth Aspect Ratio) a partir de razones geométricas entre landmarks. El modelo final usa 10 de esas features por instante, concatenadas con el embedding de 64-D de una CNN ya entrenada sobre el recorte facial y con pesos congelados — el vector fusionado de 74 valores por instante, sobre una ventana de hasta 100 instantes, alimenta una LSTM(64) con salida softmax binaria.

- Justificación de algoritmos: RandomForest y la red densa sobre features de un solo frame quedaron topadas en 33–41% de accuracy — el techo lo explica la correlación de Spearman máxima de $|r|=0.26$ entre cualquier feature de un solo frame y el nivel de somnolencia. Fusionar el embedding congelado con las features geométricas y alimentarlo a una LSTM sobre una ventana llevó el resultado a 84.24% / 0.8375 F1 macro (Tabla III), confirmando que el contexto temporal revela algo que un solo frame no puede, estructuralmente, ver.

- La clasificación final no usa argmax sino un umbral de decisión ($t^*=0.57$) elegido en validación con criterio de seguridad: entre los umbrales con recall de Drowsy ≥ 0.70 , el

de mejor precisión — perder a un conductor somnoliento es el error más costoso.

- Justificación de MediaPipe: Face Detector y FaceLandmarker son arquitecturas tipo BlazeFace, ligeras y cuantizables para CPU/NPU sin GPU dedicada — la única opción evaluada con inferencia en tiempo real ya validada en hardware de borde de bajo costo.

La Fig. 2 muestra el pipeline del modelo desplegado. Sobre 1,440 ventanas de prueba de 10 sujetos nunca vistos en entrenamiento (780 Not Drowsy / 660 Drowsy): Not Drowsy alcanzó precisión 0.80 y recall 0.94 (F1 0.87); Drowsy, precisión 0.91 y recall 0.73 (F1 0.81) — el recall más bajo de las dos clases es, deliberadamente, el error que más cuesta reducir dado el criterio de seguridad de la Sección III-B.

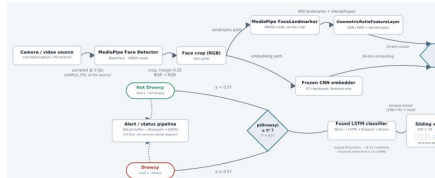


Fig. 2 — Pipeline del modelo desplegado:
MediaPipe (Face Detector + FaceLandmarker) →
CNN congelada + GeometricRatioFeatureLayer →
fusión → LSTM (t*=0.57).

IV. RESULTADOS OBTENIDOS DEL PROYECTO

Se comparó un conjunto de arquitecturas sobre el mismo problema binario: RandomForest y una red densa sobre features geométricas de un solo frame (33–41% de accuracy); una CNN sobre el recorte facial completo (59.64% accuracy, 0.5273 F1 macro); y el modelo final — el embedding congelado de esa CNN fusionado con features geométricas y alimentado a una LSTM sobre una ventana de hasta 20 segundos — que alcanzó 84.24% accuracy, 0.8375 F1 macro sobre sujetos nunca vistos, casi el doble del F1 de la CNN de un solo frame (Tabla III). El modelo final se empaquetó como un par de artefactos Keras reproducibles (la CNN congelada + la LSTM de fusión) y se integró en cv-argus: corrió de punta a punta contra una cámara en vivo y contra video grabado, con una ventana deslizante ligera manteniendo el estado entre frames. De los seis módulos planeados en el borde, dos quedaron terminados como código — model/ y pipeline/ — corriendo de punta a punta; los otros cuatro (orchestrator/, buffer/, sender/, alerts/), el firmware del

ESP32, el backend y el frontend siguen sin código. El número reportado es de un solo fold (StratifiedGroupKFold, sujetos disjuntos), sin validación cruzada todavía; no se ha corrido de punta a punta contra hardware real de Raspberry Pi (solo contra un contenedor Docker en una laptop); y la identidad exacta del checkpoint de CNN usado en producción frente al usado para generar los embeddings de entrenamiento no está verificada.

V. CONCLUSIONES Y TRABAJO A FUTURO

El desarrollo confirmó que un enfoque puramente geométrico (EAR/MAR + pose + blendshapes de un solo frame) tiene un techo real y medido — no solo sospechado — alrededor del 33–41% de accuracy, y que superarlo requirió incorporar información visual cruda (una CNN sobre el recorte facial) y, sobre todo, dar contexto temporal a esa información (fusionar el embedding de esa CNN con features geométricas y clasificar sobre una ventana, no un frame suelto). El resultado final — 84.24% de accuracy, 0.8375 de F1 macro — casi duplica el F1 de la misma CNN juzgando frame por frame, confirmando de forma directa la hipótesis central del proyecto: la duración y velocidad del cierre ocular a lo largo de una ventana es una señal que un solo instante no puede capturar estructuralmente. El diseño edge-first está además pensado para escalar por flota: sumar camiones no implica escalar un servidor central de inferencia, porque cada unidad ya trae su propio cómputo.

Trabajo a futuro:

- Validar el desempeño del modelo final corriendo en hardware real (Raspberry Pi 5): latencia de inferencia por frame, uso de CPU/memoria en ARM.
- Correr una validación cruzada (k-fold) sobre el resultado — hoy es un solo fold agrupado por sujeto.
- Confirmar que el checkpoint de la CNN que cv-argus descarga en producción es exactamente el mismo usado para generar los embeddings con los que se entrenó la LSTM final.
- Completar e integrar orchestrator/, buffer/, sender/ y alerts/ con hardware real (Pi 5 + cámara CSI) para validar la cadena completa de alerta.
- Prototipar el firmware del ESP32 sobre el diseño ya decidido (Bluetooth, polling de

la cola SQLite, HTTP hacia el backend FastAPI).

- Implementar el backend/frontend en la nube sobre FastAPI + MongoDB; OSRM como mejora futura.

- Módulos opcionales fuera del MVP: segmentación de carriles, fusión biométrica multimodal, respaldo satelital, y ampliar el conjunto de entrenamiento con más sujetos.

RECONOCIMIENTOS

Ing. Mario Antonio Ruz Canul, asesor del proyecto, por su guía a lo largo del desarrollo.

REFERENCIAS

- [1] R. Ghoddoosian, M. Galib, y V. Athitsos, “A realistic dataset and baseline temporal model for early drowsiness detection,” en *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2019.
- [2] H. Bodaghi et al., “UL-DD: A multimodal drowsiness dataset,” 2025.
- [3] M. Hidalgo-Gadea et al., “Towards better microsleep predictions in fatigued drivers: exploring benefits of personality traits and IQ,” *Ergonomics*, 2021.
- [4] E. D. Enríquez Gallegos, “Detección de somnolencia en conducción diurna y nocturna utilizando CNN MobileNet,” Trabajo de grado, Universidad Técnica del Norte, Ecuador, 2025.
- [5] Geotab Inc., “Soluciones de monitoreo del conductor.” [Online]. Available: <https://www.geotab.com/es-latam/soluciones-de-gestion-de-flotas/rastreo-conductor/>
- [6] Samsara Inc., “Soluciones de monitoreo del conductor.” [Online]. Available: <https://www.samsara.com/mx/products/platform/ai-samsara-intelligence>
- [7] Aurora Innovation, “Automatización comercial de Nivel 4.” [Online]. Available: <https://www.dufrei.com/blog/noticias-2/aurora-triplica-flota-autonoma-en-eu-y-mexico-161>
- [8] Kodiak Robotics, “Automatización comercial de Nivel 4.” [Online]. Available: <https://kodiak.ai/news/kodiak-delivers-customer-owned-autonomous-robotrucks-to-atlas>
- [9] Norma Oficial Mexicana NOM-087-SCT-2, Transporte terrestre — Condiciones físico-mecánicas y de

seguridad para la operación, Secretaría de Comunicaciones y Transportes, México.

- [10] Google, “MediaPipe Face Landmarker,” documentación técnica. [Online]. Available: https://developers.google.com/edge/mediapipe/solutions/vision/face_landmarker